

```

import pandas as pd

# Load your CSV file
df = pd.read_csv('brain_stroke.csv') # Replace with your actual file path

# Option 1: Interpolation (recommended for numerical data like 'bmi')
df['bmi'] = df['bmi'].interpolate(method='linear')

# Option 2 (alternative): Fill missing values forward then backward
# df['bmi'] = df['bmi'].fillna(method='ffill').fillna(method='bfill')

# Save cleaned file (optional)
df.to_csv('cleaned_brain_stroke.csv', index=False)

import pandas as pd
import numpy as np
import os
import pickle
import zipfile
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, mean_squared_error, r2_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# Load data
df = pd.read_csv("cleaned_brain_stroke.csv")

# Fill missing BMI values using interpolation
df['bmi'] = df['bmi'].interpolate(method='linear')

# Encode categorical variables
le = LabelEncoder()
for col in ['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status']:
    df[col] = le.fit_transform(df[col].astype(str))

# Split features and target
X = df.drop('stroke', axis=1)
y = df['stroke']

# Scale numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42, stratify=y)

# Save test sets for API evaluation
pd.DataFrame(X_test, columns=X.columns).to_csv("X_test.csv", index=False)
pd.DataFrame(y_test).to_csv("y_test.csv", index=False)

# Models to train
models = {
    "logistic_regression": LogisticRegression(max_iter=1000),
    "random_forest": RandomForestClassifier(),
    "naive_bayes": GaussianNB(),
    "svm": SVC(probability=True),
    "knn": KNeighborsClassifier()
}

# Save scaler and label encoder
with open("scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)

with open("label_encoder.pkl", "wb") as f:
    pickle.dump(le, f)

# Train and evaluate

```

```

metrics = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    # Save model
    with open(f"{name}_model.pkl", "wb") as f:
        pickle.dump(model, f)

    # Calculate metrics
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    metrics.append({
        "Model": name,
        "Accuracy": acc,
        "Precision": prec,
        "Recall": recall,
        "F1 Score": f1,
        "MSE": mse,
        "R2 Score": r2
    })

# Save metrics
metrics_df = pd.DataFrame(metrics)
metrics_df.to_csv("model_performance_metrics.csv", index=False)

# Save metrics.pkl
with open("model_metrics.pkl", "wb") as f:
    pickle.dump(metrics, f)

# Zip all model files
with zipfile.ZipFile("trained_models_bundle.zip", 'w') as zipf:
    files_to_zip = [
        "X_test.csv", "y_test.csv", "scaler.pkl", "label_encoder.pkl",
        "model_performance_metrics.csv", "model_metrics.pkl"
    ] + [f"{name}_model.pkl" for name in models.keys()]
    for file in files_to_zip:
        zipf.write(file)

print("Training complete. All models and files saved to 'trained_models_bundle.zip'.")

```

 Training complete. All models and files saved to 'trained_models_bundle.zip'.

```

import pickle
import pandas as pd
import numpy as np

# Example input
data_input = {
    'gender': 'Male',
    'age': 67,
    'hypertension': 0,
    'heart_disease': 1,
    'ever_married': 'Yes',
    'work_type': 'Private',
    'Residence_type': 'Urban',
    'avg_glucose_level': 228.69,
    'bmi': 36,
    'smoking_status': 'formerly smoked'
}

# Convert to DataFrame
input_df = pd.DataFrame([data_input])

# Load LabelEncoder and transform categorical variables
with open("label_encoder.pkl", "rb") as f:
    le = pickle.load(f)

```

```

for col in ['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status']:
    input_df[col] = le.fit_transform(input_df[col].astype(str)) # Ensure match with training encoding

# Load Scaler
with open("scaler.pkl", "rb") as f:
    scaler = pickle.load(f)

# Scale inputs
scaled_input = scaler.transform(input_df)

# Load models
model_files = [
    "logistic_regression_model.pkl",
    "random_forest_model.pkl",
    "naive_bayes_model.pkl",
    "svm_model.pkl",
    "knn_model.pkl"
]

# Predict with each model
for model_file in model_files:
    with open(model_file, "rb") as f:
        model = pickle.load(f)
    prediction = model.predict(scaled_input)[0]
    result = "Stroke Risk" if prediction == 1 else "No Stroke Risk"
    print(f"Model: {model_file.replace('_model.pkl', '')} => Prediction: {result}")

➡ Model: logistic_regression => Prediction: No Stroke Risk
Model: random_forest => Prediction: No Stroke Risk
Model: naive_bayes => Prediction: Stroke Risk
Model: svm => Prediction: No Stroke Risk
Model: knn => Prediction: No Stroke Risk

import pickle
import pandas as pd
import numpy as np

# Example input
data_input = {
    'gender': 'Male',
    'age': 67,
    'hypertension': 0,
    'heart_disease': 1,
    'ever_married': 'Yes',
    'work_type': 'Private',
    'Residence_type': 'Urban',
    'avg_glucose_level': 228.69,
    'bmi': 36,
    'smoking_status': 'formerly smoked'
}

# Convert to DataFrame
input_df = pd.DataFrame([data_input])

# Load LabelEncoder and transform categorical variables
with open("label_encoder.pkl", "rb") as f:
    le = pickle.load(f)

for col in ['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status']:
    input_df[col] = le.fit_transform(input_df[col].astype(str)) # Ensure match with training encoding

# Load Scaler
with open("scaler.pkl", "rb") as f:
    scaler = pickle.load(f)

# Scale inputs
scaled_input = scaler.transform(input_df)

# Load model performance metrics
metrics_df = pd.read_csv("model_performance_metrics.csv")

# Load models
model_files = [

```

```

"logistic_regression_model.pkl",
"random_forest_model.pkl",
"naive_bayes_model.pkl",
"svm_model.pkl",
"knn_model.pkl"
]

# Predict and show metrics
print("\nPrediction Results with Metrics:\n")
for model_file in model_files:
    model_name = model_file.replace("_model.pkl", "")
    with open(model_file, "rb") as f:
        model = pickle.load(f)

    prediction = model.predict(scaled_input)[0]
    result = "Stroke Risk" if prediction == 1 else "No Stroke Risk"

# Fetch metrics
metrics = metrics_df[metrics_df['Model'] == model_name].to_dict('records')
if metrics:
    metrics_info = metrics[0]
    print(f"Model: {model_name} => Prediction: {result}")
    print(f"  Accuracy: {metrics_info['Accuracy']:.2f}")
    print(f"  Precision: {metrics_info['Precision']:.2f}")
    print(f"  Recall: {metrics_info['Recall']:.2f}")
    print(f"  F1 Score: {metrics_info['F1 Score']:.2f}")
    print(f"  MSE: {metrics_info['MSE']:.2f}")
    print(f"  R2 Score: {metrics_info['R2 Score']:.2f}\n")
else:
    print(f"Model: {model_name} => Prediction: {result} (Metrics not found)\n")

# ---- Highlight Best Models Based on Recall and F1 Score ----

# Sort by Recall (most important for medical detection)
ranked_metrics = metrics_df.sort_values(by=["Recall", "F1 Score"], ascending=False).reset_index(drop=True)

print("\n🏆 Ranked Models by Recall & F1 Score (for Stroke Detection):\n")
for i, row in ranked_metrics.iterrows():
    highlight = "👑 Top Choice" if i == 0 else ""
    print(f"Rank {i+1}: {row['Model']}")
    print(f"  Accuracy: {row['Accuracy']:.2f}")
    print(f"  Precision: {row['Precision']:.2f}")
    print(f"  Recall: {row['Recall']:.2f}")
    print(f"  F1 Score: {row['F1 Score']:.2f}")
    print(f"  MSE: {row['MSE']:.2f}")
    print(f"  R2 Score: {row['R2 Score']:.2f} {highlight}\n")

🔄 Accuracy: 0.95
Precision: 0.00

```

```

F1 Score: 0.00
MSE: 0.05
R2 Score: -0.05

```

```

Rank 3: random_forest
Accuracy: 0.95
Precision: 0.00
Recall: 0.00
F1 Score: 0.00
MSE: 0.05
R2 Score: -0.05

```

```

Rank 4: svm
Accuracy: 0.95
Precision: 0.00
Recall: 0.00
F1 Score: 0.00
MSE: 0.05
R2 Score: -0.05

```

```

Rank 5: knn
Accuracy: 0.94
Precision: 0.00
Recall: 0.00
F1 Score: 0.00
MSE: 0.06
R2 Score: -0.16

```

```

import pickle
import pandas as pd
import numpy as np
import requests

```

```

# Function to get user input

```

```

def get_input():
    print("Enter the following details:")

    gender = input("Gender (Male/Female): ")
    age = int(input("Age: "))
    hypertension = int(input("Hypertension (0: No, 1: Yes): "))
    heart_disease = int(input("Heart Disease (0: No, 1: Yes): "))
    ever_married = input("Ever Married (Yes/No): ")
    work_type = input("Work Type (Private/Government/Self-employed/Children): ")
    residence_type = input("Residence Type (Urban/Rural): ")
    avg_glucose_level = float(input("Average Glucose Level: "))
    bmi = float(input("BMI: "))
    smoking_status = input("Smoking Status (never smoked/formerly smoked/smokes): ")

```

```

# Return as a dictionary

```

```

return {
    'gender': gender,
    'age': age,
    'hypertension': hypertension,
    'heart_disease': heart_disease,
    'ever_married': ever_married,
    'work_type': work_type,
    'Residence_type': residence_type,
    'avg_glucose_level': avg_glucose_level,
    'bmi': bmi,
    'smoking_status': smoking_status
}

```

```

# Get input from user

```

```

data_input = get_input()

```

```

# Convert to DataFrame

```

```

input_df = pd.DataFrame([data_input])

```

```

# Load LabelEncoder and transform categorical variables

```

```

with open("label_encoder.pkl", "rb") as f:
    le = pickle.load(f)

```

```

for col in ['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status']:
    input_df[col] = le.fit_transform(input_df[col].astype(str)) # Ensure match with training encoding

```

```

# Load Scaler

```

```

with open("scaler.pkl", "rb") as f:
    scaler = pickle.load(f)

# Scale inputs
scaled_input = scaler.transform(input_df)

# Prepare the data to send to the API (convert to dictionary)
data_to_send = {
    'gender': data_input['gender'],
    'age': data_input['age'],
    'hypertension': data_input['hypertension'],
    'heart_disease': data_input['heart_disease'],
    'ever_married': data_input['ever_married'],
    'work_type': data_input['work_type'],
    'Residence_type': data_input['Residence_type'],
    'avg_glucose_level': data_input['avg_glucose_level'],
    'bmi': data_input['bmi'],
    'smoking_status': data_input['smoking_status']
}

# Send data to Render Flask API
api_url = "https://brain-stroke-prediction-80dp.onrender.com/predict"
response = requests.post(api_url, json=data_to_send)

# Check if the request was successful
if response.status_code == 200:
    prediction_data = response.json()

    # Display prediction results and metrics
    print("\nPrediction Results with Metrics:\n")
    for model_name, result in prediction_data['predictions'].items():
        print(f"Model: {model_name} => Prediction: {result}")

    # Final Recommendation
    print(f"\nFinal Recommendation: {prediction_data['final_recommendation']}")

    # Model Metrics
    print("\nModel Metrics:")
    for metric in prediction_data['model_metrics']:
        print(f"Model: {metric['Model']} - Accuracy: {metric['Accuracy']:.2f}, "
              f"F1 Score: {metric['F1 Score']:.2f}, Precision: {metric['Precision']:.2f}, "
              f"Recall: {metric['Recall']:.2f}")
else:
    print(f"Error in API call: {response.status_code} - {response.text}")

```

↔ Enter the following details:

Gender (Male/Female): Male
 Age: 67
 Hypertension (0: No, 1: Yes): 0
 Heart Disease (0: No, 1: Yes): 1
 Ever Married (Yes/No): Yes
 Work Type (Private/Government/Self-employed/Children): Private
 Residence Type (Urban/Rural): Urban
 Average Glucose Level: 228.69
 BMI: 36
 Smoking Status (never smoked/formerly smoked/smokes): formerly smoked

Prediction Results with Metrics:

Model: KNN => Prediction: No Stroke Risk
 Model: Logistic Regression => Prediction: No Stroke Risk
 Model: Naive Bayes => Prediction: Stroke Risk
 Model: Random Forest => Prediction: No Stroke Risk
 Model: SVM => Prediction: No Stroke Risk

Final Recommendation: Best performing model: logistic_regression with accuracy 0.95

Model Metrics:

Model: logistic_regression - Accuracy: 0.95, F1 Score: 0.00, Precision: 0.00, Recall: 0.00
 Model: random_forest - Accuracy: 0.95, F1 Score: 0.00, Precision: 0.00, Recall: 0.00
 Model: naive_bayes - Accuracy: 0.86, F1 Score: 0.20, Precision: 0.14, Recall: 0.34
 Model: svm - Accuracy: 0.95, F1 Score: 0.00, Precision: 0.00, Recall: 0.00
 Model: knn - Accuracy: 0.94, F1 Score: 0.00, Precision: 0.00, Recall: 0.00

